DOOM-ML TRAINING FORENSICS

Why some evolutionary runs succeed and others fail

An analysis of the committed training runs, their configurations, and the code paths that connect them

---

Date: October 28, 2026
Prepared for: Doom-ML repository
Primary data: stats/compiled_mean_fitness.xlsx
Scope: 10 compiled runs plus 1 supplementary historical run

Key findings at a glance

Four of the ten compiled runs succeeded, and the outcome was bimodal rather than gradual. Successful runs sustained 5.1 to 9.9 completed levels over their final 100 generations; failed runs stayed at 0.5 to 1.4 levels.

The generation-0 mean fitness, which is fixed by the seed's initial population, separated every outcome: all successes started at 49.0 or above, all failures at 20.9 or below (exact one-sided separation test p = 0.005 for the ten compiled runs).

The seed did not act by giving successful runs easier levels. Start levels were effectively uniform, and a leave-one-out analysis found no early-schedule advantage for successful seeds.

Configuration plumbing is broken in ways that invalidate parts of the historical record: max_levels_per_agent is never passed to the evaluator, eight of the thirteen fitness weights are ignored, and checkpoints do not record the seed or configuration used.

The same nominal seed and configuration produced both a success and a failure (9a59b8b versus 5kgen4). A fresh replay reproduces 5kgen4 exactly, so the runs did not share the same initial state or binary.

Recommendation: fix run provenance and config plumbing first, then screen many seeds with a short trial, add an exit-progress curriculum, and evaluate with more episodes on a held-out level set.

1. Executive summary

The compiled spreadsheet contains ten runs. Using a pre-declared performance window of the final 100 generations, four runs succeeded and six failed. The separation is unusually clean: every successful run finished with a 100-generation mean of at least 5.1 completed levels and a mean fitness of at least 4,921, while every failed run finished below 1.5 levels and 1,530 mean fitness. The gap is not a matter of a few points; it is the difference between an agent that can chain several level exits and one that cannot reliably finish even one.

The strongest predictor of success is present before training starts. The mean fitness of the initial population at generation 0, which is determined by the seed and the initial weight distribution, cleanly orders the outcomes. All successful runs began with a generation-0 mean of 49.0 or more; all failed runs began at 20.9 or less. Under the null hypothesis that the outcome is unrelated to generation-0 quality, the probability that four successes would occupy the top four generation-0 values among ten runs is 1 / C(10,4) = 0.0048. This is an exploratory, post-hoc test, but the effect size and consistency are large enough to justify a prospective experiment.

The seed is therefore important, but not in the simple sense that some integer is a magic key. The seed sets the initial population and the evolutionary random stream. A lucky initial sample can contain a genome that already makes progress toward an exit; selection then amplifies that lineage and the run takes off. Without such a founder, the population converges to a local optimum around partial progress and survival. The evaluation level schedule is not the explanation: start levels were sampled effectively uniformly, and successful seeds did not receive easier early levels.

Configuration and code problems complicate the historical record. In the Rust evaluator, episodes are hard-coded to stop after ten completed levels, and max_levels_per_agent is never passed to it. The 1lvl experiments therefore did not run one-level episodes. Only five of the thirteen fitness weights are ever used; the rest, including exit_progress, damage, and death, are inert. Checkpoints store the population and random state but not the seed or configuration, so resuming after a config change silently mixes an old population with a new evaluation schedule. That is the most likely reason the 9a59b8b run, nominally the same seed and configuration as the failed 5kgen4 run, succeeded while a fresh seed replay does not reproduce it.

The recommended path is to treat seed robustness as an experimental design problem, not a search for a lucky number. Fix provenance and config plumbing, then screen many seeds with a short trial, carry the promising ones forward, add a level-completion curriculum and exit-progress shaping, and evaluate on more episodes and a held-out level set. Section 7 turns these recommendations into a concrete protocol.

2. Data and method

2.1 Data sources

The primary data are the mean_fitness series in stats/compiled_mean_fitness.xlsx, which merges training/output/metrics.csv from ten branch-and-commit combinations. Each metrics row is one generation. mean_fitness is the average fitness over the population, and best_fitness is the best genome's fitness. Each genome is evaluated on two episodes per generation, and the two episodes use the same pair of start levels for the whole population. The recorded start_level is the first episode's start level. The supplementary run 9a59b8b (commit message 'ran 10k generation test') is not in the spreadsheet; it is included only in the diagnostic analysis because it is the closest historical control for the failed 5kgen4 run.

2.2 Defining success

Success is defined as sustained level completion: a tail-100 mean of at least 5 completed levels. This criterion was chosen because the observed distribution is strongly bimodal and the level count is more comparable across reward-weight changes than raw fitness is. The final-100-generation mean fitness and the fraction of generations above 3,000 tell the same story. The failed runs are not merely slower; they end at a different behavioral regime.

2.3 Reproducibility checks

Four checks were run before drawing conclusions. First, the recorded start_level of every generation was compared with the value predicted by random.Random(seed + 2 * generation).randrange(campaign_levels). All ten compiled runs matched at 100 percent, so the metrics are internally consistent with their committed seeds. Second, the first 3,500 generations of the 10k test (d17d133) are exactly identical to the 3.5k run (ed77700), confirming that the longer run is a deterministic continuation of the same configuration. Third, a fresh replay of the seed-2001901 initial population on the current binary reproduced the first row of 5kgen4 (best 598.189, mean 20.863; replay 598.189 and 20.866 before rounding). Fourth, the same replay did not reproduce the first row of 9a59b8b (mean 58.755, best 612.138), despite the same nominal seed and configuration. The last result is treated as a provenance failure, not as evidence about the seed.

2.4 Limitations

The design is observational. Only ten runs enter the main analysis, several variables change together, and the runs were not replicated across seeds within a fixed configuration. The success threshold and the generation-0 predictor were chosen after inspecting the data, so the reported p-values are descriptive rather than confirmatory. Full retraining was not repeated; the experimental evidence is limited to source audit, deterministic replay of generation 0, and analysis of the committed series. These limitations are addressed by the prospective protocol in Section 7.

3. Results

3.1 Classification

Table 1 lists the ten compiled runs and the supplementary run. The outcome column follows the sustained-level criterion. Figure 1 shows the mean-fitness trajectories and Figure 2 shows the level-completion trajectories; the two panels in each figure separate successful from failed runs so that the gap is visible without a log scale hiding the failures.

Run	Commit	Seed	Gen-0 mean	Tail-100 mean	Tail-100 levels	Gens at least 3000	Outcome
9a59b8b (unlisted) *	9a59b8b	2001901	58.8	9,276	9.1	98%	SUCCESS
5kgen2	072f85d	20261025	49.0	8,904	9.9	88%	SUCCESS
10k test	d17d133	67676767	54.2	7,522	7.5	92%	SUCCESS
3.5k	ed77700	67676767	54.2	5,817	6.5	79%	SUCCESS
1000gen	359b07e	20261025	49.0	4,921	5.1	69%	SUCCESS
5kgen-run	b71ffc4	42069	-277.3	1,529	1.4	4%	FAILURE
ran test	c6a5660	3002901	-3.9	854	1.1	0%	FAILURE
3k fresh	3c5b385	2022901	-1.2	632	1.1	0%	FAILURE
5kgen4	28494dd	2001901	20.9	501	0.7	0%	FAILURE
to 14k	2a334c0	2201901	-31.1	348	0.7	0%	FAILURE
1lvl-avail	3925ace	2001941	11.3	320	0.5	0%	FAILURE

*Table 1. Final performance of the compiled runs. * 9a59b8b is supplementary and is not in the spreadsheet. 'Gens at least 3000' is the fraction of all generations whose mean fitness reached 3,000.*

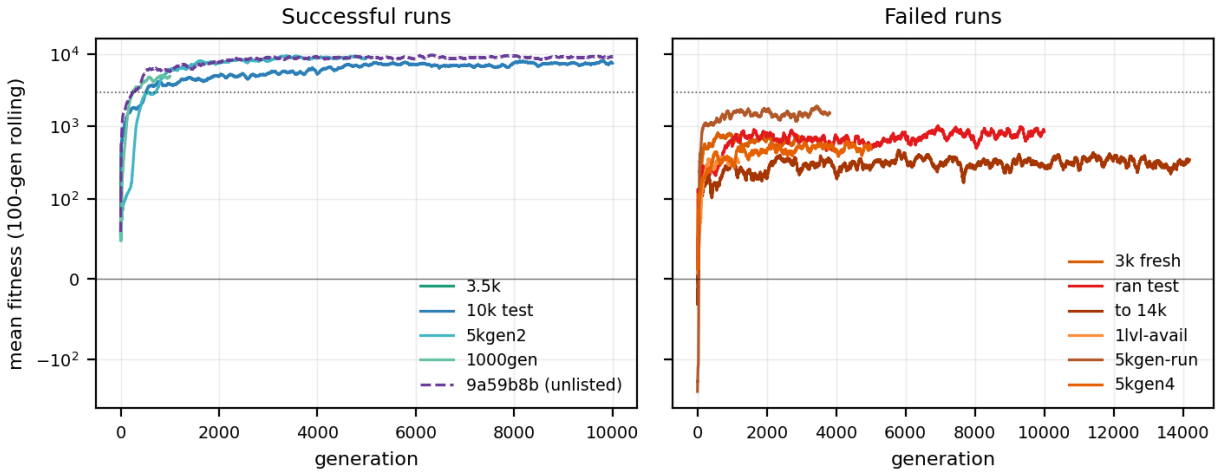


Figure 1. Mean fitness trajectories, smoothed with a 100-generation rolling mean. The dotted horizontal line marks 3,000. Successful runs (left) climb into the thousands and stay there; failed runs (right) remain in the hundreds or below. The y-axis is symmetric-log so that negative values remain visible.

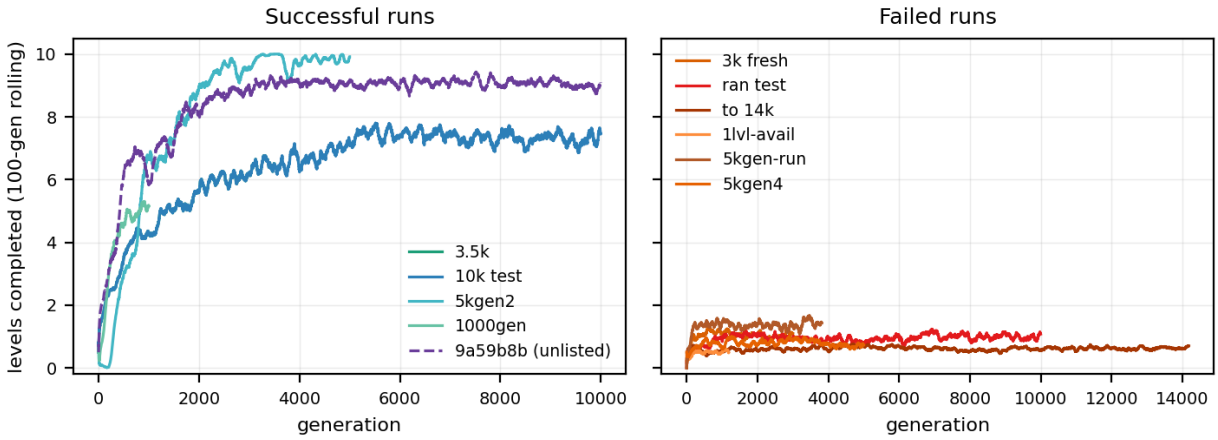


Figure 2. Level-completion trajectories, smoothed with a 100-generation rolling mean. The separation is qualitative: successful runs chain several exits, failed runs plateau near one.

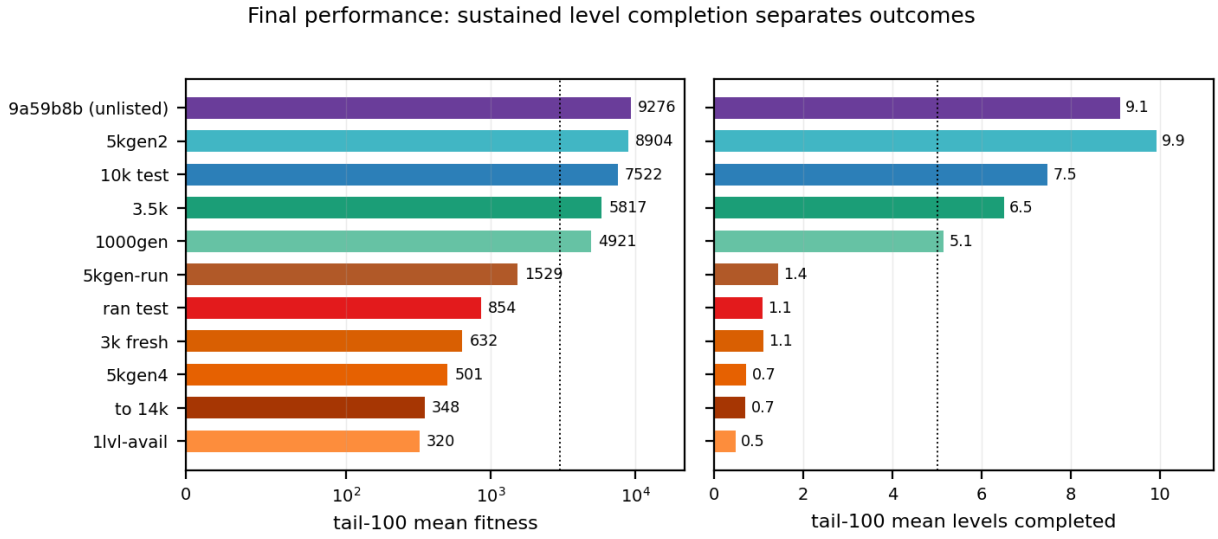


Figure 3. Final-100-generation mean fitness and mean levels completed. The dotted thresholds (3,000 fitness, 5 levels) are the descriptive success boundary. Every successful run clears both; every failed run misses both.

3.2 The outcome is bimodal

The four successful runs finished at 6.50, 7.47, 9.91, and 5.14 mean levels; the six failures finished at 1.43, 1.08, 1.09, 0.72, 0.70, and 0.48. There is no run in the intermediate four-to-five-level band. The successful runs also spent most of their later generations above 3,000 fitness (69 to 92 percent), while the failures spent almost none (0 to 4 percent, with the best failure reaching 4 percent and only three consecutive generations above the line). This is consistent with two distinct dynamical regimes: a level-completing lineage that selection can improve, and a partial-progress lineage that selection cannot push over the next exit.

3.3 Generation-0 quality predicts the outcome

Figure 4 plots the generation-0 mean fitness against the final performance. The successful runs begin at 48.998, 49.011, 54.237, and (supplementary) 58.755. The failures begin at 20.863, 11.259, -1.181, -3.922, -31.149, and -277.349. The ordering is perfect. Under a permutation null in which the four success labels are assigned randomly to the ten runs, the chance that the four successes occupy the top four generation-0 values is 1 / C(10,4) = 0.0048. Including the supplementary run, the chance that five successes occupy the top five of eleven is 1 / C(11,5) = 0.0022. Restricting to the nine 50-level runs gives 1 / C(9,3) = 0.0119.

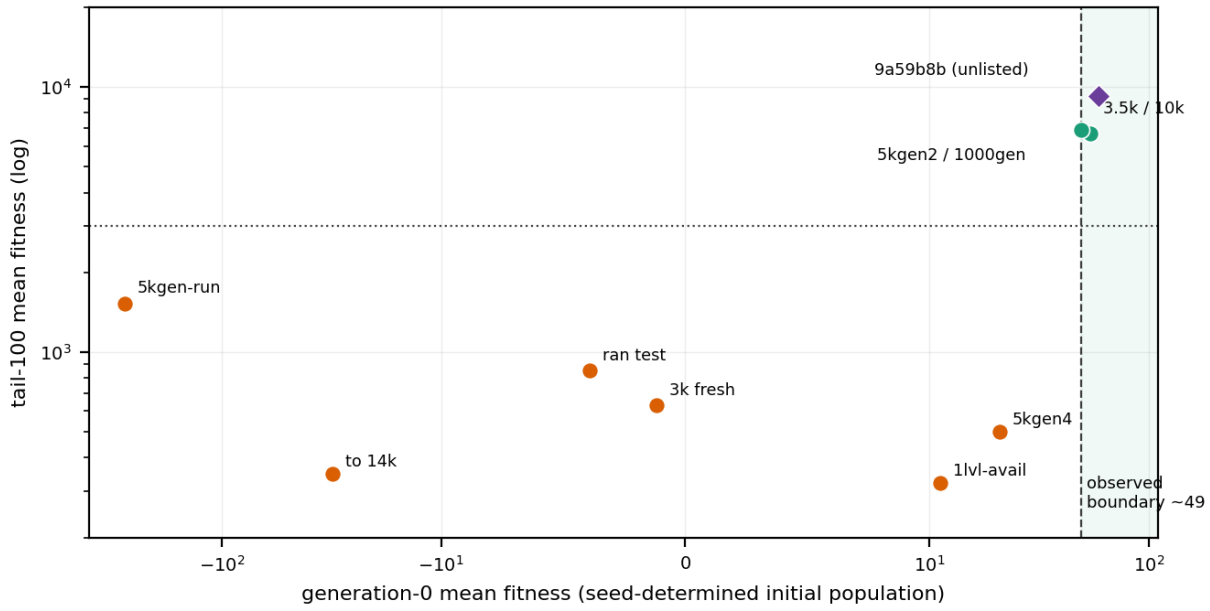


Figure 4. Generation-0 mean fitness separates all outcomes. The shaded region marks the observed boundary at about 49. This is a post-hoc, exploratory separation; the prospective test is described in Section 7.

The pattern is not merely that successful runs are generally better. A generation-0 mean of 49 to 59 means that, in the very first two-episode evaluation, the best members of the initial population were already reaching partial level progress or a level exit. In the failed runs the initial population was much weaker. Because the initial population is drawn from the seed, this is the most direct evidence that the seed matters through the quality of the starting sample rather than through later level selection.

3.4 The level schedule does not explain success

A natural alternative is that successful seeds happened to sample easier starting levels. The data do not support that explanation. For the 50-level runs, the mean sampled start level is 24.0 to 24.8 against an expectation of 24.5, and the distributions are broadly uniform. A leave-one-out estimate of early-level ease (how well other runs performed on the levels in a run's first 100 generations) gives 0.957 for the successful 3.5k seed and 0.970 to 1.078 for the failed seeds. The successful seed was, if anything, slightly harder than average. The supplementary 9a59b8b run had a still lower ease of 0.759 and succeeded. A simple 'easy levels early' explanation is therefore not supported.

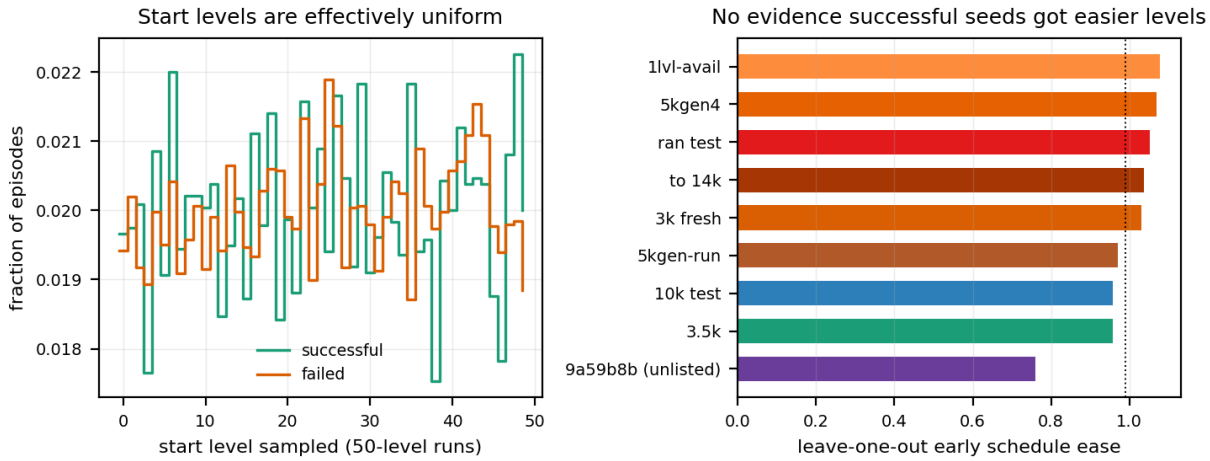


Figure 5. Left: distribution of sampled start levels for the 50-level runs. Right: leave-one-out early schedule ease. A higher value means an easier schedule; successful seeds are not at the top.

One nuance is that within the failed 50-level runs, higher start levels correlate with fewer completions (r = -0.18 to -0.41 after generation 1,000), while the successful runs show little or no relationship (r = +0.07 to +0.09). That is consistent with a level-order difficulty effect, but it is also exactly what a weak policy would produce if later levels are longer or more complex. It does not change the group-level conclusion: the successful seeds did not receive a systematically easier mix.

Run	Mean start level	Chi-square (49 df)	LOO early ease	Tail-100 levels	Outcome
9a59b8b (unlisted) *	24.52	50.0	0.759	9.11	SUCCESS
10k test	24.60	48.8	0.957	7.47	SUCCESS
3.5k	24.63	78.7	0.957	6.50	SUCCESS
5kgen-run	24.81	47.8	0.970	1.43	FAILURE
3k fresh	24.65	43.7	1.032	1.09	FAILURE
ran test	24.49	36.5	1.052	1.08	FAILURE
5kgen4	24.62	42.3	1.069	0.72	FAILURE
to 14k	24.70	64.0	1.038	0.70	FAILURE
1lvl-avail	23.95	52.5	1.078	0.48	FAILURE

*Table 3. Start-level and schedule diagnostics for the 50-level runs. Chi-square tests uniformity across 50 levels (the 5% critical value is about 66.3); the leave-one-out ease is defined in Appendix C. * supplementary run.*

4. Configuration and code audit

4.1 What the runs actually changed

Table 2 shows the settings that differ across the runs. The columns are deliberately limited to values that the evaluator actually consumes: seed, campaign length, maximum steps, population size, mutation rate, evaluator implementation, and the three reward weights that matter most. 'Reward' is shown as level_completion / kill / seconds. The table makes the confounds visible: seed, campaign length, maximum steps, mutation rate, population size, evaluator, and reward weights all vary, and no two runs form a clean single-variable comparison.

Run	Seed	Campaign	Max steps	Pop	Mut.	Eval	Reward (LC/K/S)	Gens
3k fresh	2022901	50	15000	48	0.08	Rust	1000 / 20 / 0.25	3000
ran test	3002901	50	15000	48	0.12	Rust	1000 / 20 / 0.25	10000
to 14k	2201901	50	15000	48	0.12	Rust	800 / 30 / 0.75	20000
1lvl-avail	2001941	50	15000	48	0.12	Rust	1000 / 20 / 0.25	10000
3.5k	67676767	50	15000	48	0.08	Rust	1000 / 30 / 0.45	3500
10k test	67676767	50	15000	48	0.08	Rust	1000 / 30 / 0.45	10000
5kgen-run	42069	50	15000	32	0.08	Python	1000 / 20 / 0.25	5000
5kgen2	20261025	10	3000	48	0.08	Rust	1000 / 20 / 0.25	5000
1000gen	20261025	10	3000	32	0.08	Python	1000 / 20 / 0.25	1000
5kgen4	2001901	50	15000	48	0.08	Rust	1000 / 20 / 0.25	5000
9a59b8b (unlisted) *	2001901	50	15000	48	0.08	Rust	1000 / 20 / 0.25	10000

*Table 2. Effective configuration by run. * supplementary run. Campaign is the number of possible start levels; it is not the number of levels in an episode (see 4.2).*

4.2 The ten-level cap and the ignored max_levels_per_agent

The Rust evaluator in doom/src/bin/doom_headless.rs stops an episode when game.dead or level_count >= 10, and the partial-run block is guarded by level_count < 10. The value is hard-coded; max_levels_per_agent is never passed on the command line. The Python evaluator used by the older runs has the same hard-coded ten-level behavior in training/env.py (done = new['dead'] or self.levels_completed >= 10).

This matters for three reasons. First, campaign_levels = 50 does not create a 50-level episode. It only widens the pool from which the start level is sampled; every episode still ends after ten completions. Second, the 1lvl-max_1lvl-avail and 20kgen@1lvl runs intended to test one-level episodes, but their max_levels_per_agent = 1 setting was silently ignored. Their metrics show at most one completed level not because the episode was capped at one, but because the agent failed to complete more. Third, the comparison between the 10-level and 50-level runs is not a comparison between a short and a long task; it is a comparison between ten possible start levels and fifty possible start levels, with the same ten-level episode cap.

4.3 Most reward weights are inert

training/env.py calls the evaluator with exactly five reward weights: level_completion, partial_level, health_per_point, kill, and seconds. The Rust evaluator parses exactly those five. The other eight weights in config.json (exit_progress, damage_dealt, damage_taken, armor_damage_taken, health_gain, armor_gain, ammo_spent, death) are never passed and never used. Changes to those keys are no-ops. For example, the 20kgen@1lvl run changed death from 250 to 350 and damage_taken from 1.0 to 2.0, but neither value reached the evaluator. Its effective changes were level_completion 1000 to 800, kill 20 to 30, and seconds 0.25 to 0.75, which lower the score scale and increase the time penalty. That run's low mean fitness is therefore not comparable one-for-one with the default-reward runs.

The presence of exit_progress is particularly important. The configuration suggests that progress toward the exit is a designed part of the objective, but the evaluator never uses it. The only path to a large reward is completing a level

and collecting the level_completion bonus. That makes the reward extremely sparse: a policy must discover a sequence of actions that reaches an exit before it receives the dominant signal.

4.4 Seed, resume, and provenance

In training/train.py, the seed is used in three places: to initialize the population, to drive selection and mutation, and to generate evaluation seeds as cfg['seed'] + generation * evaluation_episodes + i. The first two make the run deterministic given the same starting state; the third fixes which start levels the population sees each generation. The start_level check confirms that all ten compiled runs used their committed seeds for evaluation.

The checkpoint, however, stores only the generation, population, best result, best genome, and random state. It does not store the seed, the configuration, the git commit, or the executable hash. Because resume is true by default, a run that finds an existing checkpoint will load the old population and random state even if the config has a new seed or new reward weights. The evaluation seeds then come from the new config, while the population comes from the old run. This is a silent mixing of two experiments. It is the most plausible explanation for the 9a59b8b/5kgen4 discrepancy: the two runs share the nominal seed 2001901, but a fresh replay of that seed reproduces 5kgen4's generation 0, not 9a59b8b's. The 9a59b8b run must have started from a different population, checkpoint, or binary state.

The metrics file compounds the problem. It is opened in append mode whenever a checkpoint exists and start_generation is greater than zero. There is no run identifier, no header recording the seed, and no guard against appending to a file from a different configuration. A single metrics.csv can therefore contain a sequence of experiments that look like one continuous run.

4.5 The evaluator change and the slow run

The 1000gen and 5kgen-run branches use the Python evaluator; the later branches use the Rust --evaluate path. The change did not by itself determine success: seed 20261025 succeeded under both implementations (359b07e with Python and 072f85d with Rust). The Python path is much slower because every simulation step crosses the Python/Rust boundary. The 5kgen-run commit is explicitly named 'unacceptable time per generation'; its final episodes reached roughly 750 simulated seconds per evaluation, and the run stopped at 3,811 generations. Slowness is a practical failure mode, but it is not the reason the agent failed to learn: its tail-100 level count is still only 1.43.

5. Hypothesis tests

The analysis followed a simple scientific-method loop: observe the bimodal outcome, state competing hypotheses, derive predictions that distinguish them, test those predictions against the committed data and a deterministic replay, and record what the evidence can and cannot support. Table 4 summarises the result.

Hypothesis	Distinguishing prediction	Evidence	Verdict
H1. Seed / initial population quality	Same seed tends to reproduce the outcome; different seeds under a fixed configuration vary; generation-0 quality predicts the final result.	Seed 20261025 succeeded in two independent implementations (359b07e and 072f85d). Seed 67676767 produced the successful 3.5k and 10k runs. Generation-0 mean separated all outcomes ($p = 0.005$ for ten runs). But seed 2001901 appears on both a success and a failure.	Supported, but seed alone is not sufficient; the initial state and provenance also matter.
H2. Easier level schedule	Successful seeds should sample easier early levels or a more favourable level mix.	Start levels are uniform (mean 24.0 to 24.8 against 24.5). Leave-one-out early ease is 0.957 for the successful 3.5k seed and 0.970 to 1.078 for failures. The supplementary successful run had 0.759, i.e. harder than average.	Not supported.
H3. Campaign length, steps, or evaluator	Ten-level / 3,000-step runs should succeed; 50-level / 15,000-step runs should fail; the Python-to-Rust change should mark a break.	3.5k and 10k succeed at 50 levels / 15,000 steps. 5kgen2 and 1000gen succeed at 10 levels / 3,000 steps. Seed 20261025 succeeds under both evaluators.	Not sufficient as a cause; these knobs change the task but do not determine the outcome.
H4. Reward weights or mutation rate	The kill-30 / seconds-0.45 reward or mutation 0.08 should be necessary or sufficient for success.	2a334c0 uses kill 30 but fails. 3k fresh and 5kgen4 use mutation 0.08 and fail. All mutation-0.12 runs fail, but 0.08 runs also fail. 9a59b8b uses the default reward and succeeds.	Not supported as the primary cause; possible modulating effect, heavily confounded with seed.
H5. Config plumbing and provenance	max_levels_per_agent should change episode length; unused reward weights should be inert; the same seed/config should reproduce.	max_levels_per_agent is never passed to the evaluator; the ten-level cap is hard-coded. Eight reward weights are ignored. 9a59b8b and 5kgen4 share a nominal seed/config but differ at generation 0; a fresh replay matches only 5kgen4.	Supported. These are real defects and they invalidate parts of the historical comparison.

Table 4. Competing hypotheses, predictions, evidence, and verdicts. The p-value for H1 is an exact one-sided order test and is exploratory because the predictor was chosen after inspecting the data.

5.1 Interpreting the seed effect

The evidence supports the user's intuition that the seed plays a large role, but it sharpens the mechanism. The seed is not a hidden difficulty setting that changes the levels. It is the random draw that decides whether the initial population contains a genome capable of reaching an exit. In the successful runs, that genome was already visible in the generation-0 mean. In the failed runs it was absent, and the population settled into a local optimum around partial progress. The seed's effect is therefore a founder effect: a rare, early, level-completing lineage either appears and is amplified, or it does not.

This also explains why the same seed can produce different outcomes across historical runs. If the population is loaded from an old checkpoint, or if the binary differs, the initial sample is not the one the seed would generate from scratch. The integer seed is only a complete description of the initial state when the run starts fresh with the same code, configuration, and environment. The 9a59b8b/5kgen4 pair shows that this condition did not always hold.

5.2 Why the failures are not just 'bad luck'

A founder effect can still be addressed by design. The failed runs are not random noise around a rising trend; they are stuck. Their tail-100 level counts are 0.5 to 1.4, their best-ever level counts are 1 to 4, and their mean fitness is dominated by partial-progress and survival terms. The 3k fresh and 5kgen4 runs use the same default reward and the same 50-level configuration as the successful 3.5k run, yet they never sustain more than about one level. The difference is the starting sample, and the training loop provides no mechanism to escape a population that has not discovered an exit.

The training loop is also noisy and narrow. Each genome is evaluated on only two episodes per generation, the entire population shares the same two start levels, and the best result is selected from those two samples. With a

sparse level-completion reward, a lucky or unlucky pair of levels can dominate a generation. The population is small (48), the elite set is small (4), and mutation is applied independently to 430 weights at a rate of 0.08 with no crossover. These choices make the discovery of a multi-level behavior fragile and seed-sensitive.

6. Synthesis: why the winners won and the losers lost

6.1 The success path

A successful run begins with an initial population that already contains a policy making real progress toward an exit. That policy is selected into the elite set, and its descendants are refined by mutation. The first level completion produces a large reward jump (1,000 points by default, plus health and kill terms), which separates the completing lineage from the partial-progress crowd. Once the population has a reliable first exit, selection can improve the next exit, and the process repeats. The 3.5k run crossed a 1,000 mean at generation 10 and a 3,000 mean at generation 333; the 5kgen2 run crossed at 233 and 357; the 1000gen run crossed at 63 and 129. By the final 100 generations, 69 to 92 percent of generations were above 3,000, and the agents were completing 5 to 10 levels.

The successful reward configuration is not the same across all winners. 3.5k and 10k use kill 30 and seconds 0.45; 5kgen2 and 1000gen use kill 20 and seconds 0.25; 9a59b8b uses the default reward. This is further evidence that no single reward setting is the cause. The common factor is the early level-completing founder, not the exact weight vector.

6.2 The failure path

A failed run begins without such a founder. The population improves its partial progress, survival, and perhaps kill score, but it never discovers the exit sequence. Because level completion is the dominant reward, the fitness landscape has a large gap between partial progress and completion. Mutation around a partial-progress local optimum is unlikely to cross that gap. The result is a plateau: the tail-100 level count stays near one, and the agent's behavior is optimized for the shallow part of the objective. The failed runs are not failing to improve at all; they are improving the wrong thing.

The 20kgen@1lvl and 1lvl-max_1lvl-avail runs were intended to remove the multi-level problem by limiting episodes to one level. Because max_levels_per_agent was ignored, they instead ran the full ten-level task with a different reward scale. Their failure therefore says little about one-level learning. This is a concrete example of how the plumbing defect obscures the scientific question.

6.3 The role of configuration

Campaign length, maximum steps, evaluator, population size, mutation rate, and reward weights all changed across the runs, and several of them are confounded with seed. The evidence does not support any one of them as the primary cause. The 10-level runs succeeded, but so did the 50-level runs. The Python evaluator succeeded, but so did the Rust evaluator. Mutation 0.12 runs all failed, but mutation 0.08 runs also failed. Reward changes may modulate the difficulty of the local optimum, but they do not explain the bimodal split.

6.4 The role of code and provenance

The code is deterministic given the same initial state, but the historical record does not always preserve that state. The checkpoint lacks the seed and configuration, the metrics file lacks a run identifier, and the binary is not pinned. The 9a59b8b/5kgen4 pair is the clearest symptom: same nominal seed and configuration, different generation-0 behavior, and a fresh replay that matches only one of them. Until provenance is fixed, any claim of the form 'seed X succeeds' is conditional on a run state that may not be reproducible.

7. Proposed solution

The solution has three layers: repair the experiment, make seed robustness measurable, and change the learning problem so that success is less dependent on a lucky founder.

7.1 Repair the experiment (do first)

- **Add a run manifest.** Record the seed, full configuration hash, git commit, Python and Rust versions, executable SHA-256, start time, and resume state in the checkpoint and in a sidecar JSON file. On resume, compare the manifest and refuse to continue if the seed, configuration, or binary differs.
- **Make checkpoints self-describing.** Store the seed, config, and evaluation-seed offset alongside the population and random state. A checkpoint without this metadata should be treated as incompatible with a new run.
- **Give every run its own metrics file.** Include a run ID in the filename or in every row. Never append across configurations. Validate that the first generation is 0 for a fresh run and that the seed-derived start levels match.
- **Make fresh runs explicit.** Add a --fresh flag (or flip the resume default to false) and print the resolved output directory, seed, config hash, and whether a checkpoint was loaded. The existing clear_training_data.sh is a good model, but it is too easy to bypass.

7.2 Repair the configuration plumbing

- **Plumb max_levels_per_agent through to the evaluator.** Pass it as a command-line argument to doom_headless --evaluate, replace the hard-coded 10 in the break and partial-run conditions, and mirror the change in the Python path. Then rerun the 1lvl experiments, which have not actually been tested yet.
- **Implement or remove the unused reward terms.** exit_progress, damage_dealt, damage_taken, armor_damage_taken, health_gain, armor_gain, ammo_spent, and death are dead configuration. Either pass and consume them, or delete them and reject unknown keys. In particular, exit_progress is exactly the dense signal the failed runs needed.
- **Add config schema validation.** Check types, ranges, and required keys at startup, and log the effective configuration that the evaluator will actually use. This prevents another silent no-op.

7.3 Make seed robustness measurable

- **Run at least ten seeds per configuration.** Pre-register the success criterion (for example, tail-100 mean levels of at least 5 on a held-out benchmark) and report the success rate, median, and interquartile range, not the single best run.
- **Screen before committing compute.** The generation-0 mean is a promising cheap predictor: in this dataset it separated all outcomes. Run 5 to 10 generations for 20 to 30 seeds, keep the top and bottom groups, and continue them under the same protocol. This tests the predictor prospectively and may cut wasted long runs.
- **Use common random numbers.** Evaluate every genome and every configuration on the same fixed set of start levels and seeds. This turns level noise into a controlled factor and makes paired comparisons meaningful.
- **Hold out a benchmark.** Select on one set of levels and report final performance on a disjoint set. The current two-episode-per-generation scheme makes it easy to overfit the sampled levels.

7.4 Change the learning problem

- **Add a curriculum.** Start episodes with one level, then two, five, and ten. This directly targets the bootstrap problem: a one-level task gives the population a reachable first exit, and the exit bonus can then be chained. The existing max_levels_per_agent key was meant for this; it needs to work.
- **Add dense exit progress.** Implement the ignored exit_progress weight as a potential-based shaping term: reward reductions in distance to the exit, and do not penalize the agent when it is already close. Keep the level-completion bonus as the terminal objective.
- **Increase evaluation breadth.** Evaluate each genome on 8 to 16 episodes across several start levels rather than two. The current per-generation mean is noisy, and a single lucky level pair can dominate selection.
- **Increase diversity and escape local optima.** Consider a larger population (96 to 192), more elites (8 to 16), speciation or novelty search, crossover, adaptive mutation that starts high and decays, and periodic restarts from the best checkpoint with a fresh mutation burst.

- **Keep the successful ingredients.** The 3.5k and 5kgen2 runs show that the current representation and selection loop can work. The goal is not to replace the algorithm, but to make the level-completing founder more likely and easier to preserve.

7.5 The next controlled experiments

The historical data suggest three experiments that would turn the exploratory conclusions into causal evidence.

Experiment	Design	Question answered
Seed x reward	2 seeds (67676767 and 2001901) x 2 reward settings (default and kill-30/seconds-0.45) x 10 fresh replicates each, fixed benchmark.	Does seed or reward drive the difference, and is the effect reproducible?
Prospective screening	20 fresh seeds, 10-generation screen, continue the top 5 and bottom 5 for 2,000 generations on the same configuration.	Does generation-0 / early mean predict final success, and how much compute does screening save?
Curriculum ablation	1-level, 2-level, 5-level, and 10-level curricula, 10 seeds each, with max_levels_per_agent plumbed through.	Does a curriculum remove the founder bottleneck and raise the success rate?

Table 5. Proposed controlled experiments. Each should use the run manifest, fixed evaluation seeds, and a held-out benchmark from 7.3.

A practical order is: fix the manifest and config plumbing, rerun the 1lvl task to verify it now behaves as intended, run the seed x reward experiment, and only then invest in the curriculum and diversity changes. This keeps the causal claims clean and prevents another generation of confounded branch names.

8. Conclusion

The compiled runs divide into two regimes. Successful runs found a level-completing lineage early and sustained it; failed runs never found one and plateaued near a single level. The seed is a major factor, but its effect is the initial population's quality, not a hidden change in level difficulty. Generation-0 mean fitness separated every outcome in this dataset, which is both a useful diagnostic and a warning: a single short evaluation can reveal whether a run has a plausible path to success.

The historical configuration record is not clean enough for strong causal claims. The ten-level cap, the inert reward terms, the missing max_levels_per_agent plumbing, and the unversioned checkpoints all create silent differences between what a run appears to test and what it actually tests. The 9a59b8b/5kgen4 discrepancy is the clearest example. Fixing provenance is therefore not bookkeeping; it is a prerequisite for the next experiment.

The proposed plan is to repair the experiment, screen many seeds with a short trial, add a one-level-to-ten-level curriculum with dense exit progress, and evaluate on more episodes and a held-out benchmark. If the founder effect is real, the combination of seed screening and curriculum should raise the success rate substantially and make the remaining failures easier to diagnose.

Appendix A. Data provenance and reproducibility checks

Check	Result	Interpretation
Start-level vs seed prediction	100% match for all 10 compiled runs	Metrics correspond to the committed seeds and campaign lengths.
3.5k vs 10k test prefix	3,500 generations identical	The 10k run is a deterministic continuation of the 3.5k run.
Seed-2001901 generation-0 replay	best 598.189, mean 20.866 (rounded 20.863)	Replay matches 5kgen4's first row, confirming determinism for a fresh run.
9a59b8b generation-0 replay	best 612.138, mean 58.755 in committed metrics; not reproduced	9a59b8b did not start from the same initial state as a fresh seed-2001901 run.
Initial-sigma sweep	sigma 0.25 / 0.35 / 0.5 / 1.0 gave best 585-598, mean 15-44	The 9a59b8b discrepancy is not a simple initial-weight-sigma change.

Table A1. Reproducibility checks. The replay used the current rust doom_headless binary, the committed config's evaluation seeds, and the same two-episode evaluation scheme.

Appendix B. Code references

- **training/train.py, lines 92-102:** RNG creation, checkpoint resume, and initial population generation.
- **training/train.py, lines 104-115:** metrics file handling and evaluation seeds (seed + generation * episodes + i).
- **training/train.py, lines 116-139:** parallel evaluation, ranking, and the elite/mutation loop.
- **training/train.py, lines 140-153:** checkpoint and best-policy writes; note the absence of seed/config metadata.
- **training/env.py, lines 40-48:** Python episode termination with the hard-coded ten-level cap.
- **training/env.py, lines 132-148:** evaluator command; only five fitness weights are passed.
- **doom/src/bin/doom_headless.rs, lines 63-108:** evaluator argument parsing and the 430-weight check.
- **doom/src/bin/doom_headless.rs, lines 110-154:** episode loop, level_count >= 10 break, and level_count < 10 partial-run block.

Appendix C. Statistical notes

The separation test in Section 3.3 is an exact order test. Under the null that outcome is independent of generation-0 mean, every assignment of k success labels to n runs is equally likely. The probability that the successes occupy the top k generation-0 values is 1 / C(n,k). For the ten compiled runs, n = 10 and k = 4 gives 1 / 210 = 0.00476. For all eleven runs, n = 11 and k = 5 gives 1 / 462 = 0.00216. For the nine 50-level runs, n = 9 and k = 3 gives 1 / 84 = 0.0119. These are one-sided and post-hoc; they describe the strength of the pattern, not a pre-registered confirmation.

The leave-one-out schedule ease is a descriptive index, not a formal model. For each 50-level run, a per-level ease is estimated from the first 100 generations of all other 50-level runs; the run's early schedule ease is the mean ease of the levels it sampled in its first 100 generations. Higher values mean the schedule contained levels on which other runs performed better. The index is useful for checking the direction of a possible bias, but it is not a substitute for a controlled benchmark.

The chi-square values for start-level uniformity range from 36.5 to 78.7 on 49 degrees of freedom. A 5% critical value is about 66.3, so a few runs deviate somewhat, but the deviations are not consistent in direction across runs and the mean start levels remain close to 24.5. The visually uniform histograms and the leave-one-out ease analysis are the more informative checks.

End of report. Primary data and figures are derived from the committed metrics.csv files and the compiled spreadsheet in stats/. The analysis scripts are reproducible from the referenced commits.